

Conferencia 3: funciones

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-09-30

El juego de adivinar de la clase pasada funciona, pero es una pared de veinte líneas sin costuras. Si ahora quisieras un torneo al mejor de tres partidas, tendrías que copiar esas veinte líneas tres veces, y si después encontraras un error habría que arreglarlo en los tres lugares. Hoy aprendemos a ponerle nombre a un pedazo de programa para poder usarlo muchas veces sin repetirlo. Al final de la clase el juego va a caber en cuatro líneas y el torneo en seis.

1. Poner nombre a un pedazo de programa

Llevas dos clases usando funciones. `print`, `input`, `int`, `math.sqrt` y `random.randint` son funciones: les das algo entre paréntesis y hacen un trabajo. Lo único nuevo de hoy es que vamos a escribir las nuestras.

Una función se define con `def`, un nombre, unos paréntesis y dos puntos. El cuerpo va indentado, igual que el de un `if` o un `while`:

```
def saludar():  
    print("Hola, MatCom")  
  
saludar()  
saludar()
```

```
Hola, MatCom  
Hola, MatCom
```

Hay dos momentos distintos y conviene separarlos desde ahora. La **definición** es el bloque del `def`: le dice a Python qué significa ese nombre, y no ejecuta nada. La **llamada** es `saludar()`, con los paréntesis: ahí es donde el cuerpo se ejecuta de verdad. Definir una función y no llamarla nunca es perfectamente legal y no imprime nada:

```
def saludar():  
    print("Hola, MatCom")  
  
print("El programa terminó")
```

```
El programa terminó
```

Los paréntesis son obligatorios para llamar. Sin ellos no hay error, pero tampoco pasa nada útil, porque el nombre a secas se refiere a la función misma y no a su resultado:

```
def saludar():  
    print("Hola, MatCom")  
  
print(saludar)
```

```
<function saludar at 0x7c51b3c2f530>
```

Eso que imprime es la función como valor. Hoy no lo vamos a usar, pero recuérdalo: significa que una función es un dato más, como un número o un texto, y sobre esa idea está construida media biblioteca estándar.

2. Parámetros

Una función que hace siempre exactamente lo mismo sirve de poco. Los **parámetros** son variables que se escriben entre los paréntesis de la definición y que reciben un valor distinto en cada llamada:

```
def saludar(nombre):  
    print(f"Hola, {nombre}")  
  
saludar("Ana")  
saludar("Pedro")
```

```
Hola, Ana  
Hola, Pedro
```

Puede haber varios, separados por comas, y se emparejan por posición con los valores de la llamada:

```
def describir(nombre, edad):  
    print(f"{nombre} tiene {edad} años")  
  
describir("Ana", 19)  
describir("Pedro", 20)
```

```
Ana tiene 19 años  
Pedro tiene 20 años
```

Dos palabras que se confunden todo el tiempo y que vas a oír en los exámenes. **Parámetro** es el nombre que aparece en la definición: nombre, edad. **Argumento** es el valor concreto que se pasa en la llamada: "Ana", 19. El parámetro es el hueco y el argumento es lo que se le mete.

Si el número de argumentos no coincide con el de parámetros, Python se queja antes de ejecutar nada del cuerpo:

```
def describir(nombre, edad):  
    print(f"{nombre} tiene {edad} años")  
  
describir("Ana")
```

```
Traceback (most recent call last):  
  File "<cell>", line 4, in <module>  
    describir("Ana")  
    ~~~~~^  
TypeError: describir() missing 1 required positional argument: 'edad'
```

3. return

Hasta aquí nuestras funciones imprimen. Lo más útil que puede hacer una función es **devolver** un valor, para que quien la llamó haga con él lo que quiera:

```
def doble(x):
    return 2 * x

print(doble(5))
print(doble(5) + doble(3))
y = doble(doble(2))
print(y)
```

```
10
16
8
```

Mira la segunda y la tercera línea. Como `doble(5)` produce un valor, se puede sumar, guardar en una variable o pasar a otra función. Recupera la distinción de la primera clase: **una función con `return` es una expresión**, y una función que solo imprime es una instrucción.

La diferencia entre imprimir y devolver es la que más trabajo cuesta al principio, así que vamos a verla de frente. Estas dos funciones parecen hacer lo mismo:

```
def area_imprime(lado):
    print(lado * lado)

def area_devuelve(lado):
    return lado * lado

area_imprime(3)
print(area_devuelve(3))
```

```
9
9
```

La salida es idéntica. Pero intenta usarlas para algo:

```
def area_imprime(lado):
    print(lado * lado)

def area_devuelve(lado):
    return lado * lado

total = area_devuelve(3) + area_devuelve(4)
print(f"El total es {total}")

total = area_imprime(3) + area_imprime(4)
```

```
El total es 25
9
16
Traceback (most recent call last):
  File "<cell>", line 10, in <module>
    total = area_imprime(3) + area_imprime(4)
            ~~~~~^~~~~~
TypeError: unsupported operand type(s) for +: 'NoneType' and 'NoneType'
```

La que imprime no sirve para calcular. Escribe en la pantalla y no deja nada. La regla práctica: **una función que calcula algo lo devuelve; imprimir es trabajo del programa principal**. Así puedes cambiar la presentación sin tocar el cálculo.

Funciones que no devuelven nada

¿Qué valor tiene entonces `area_imprime(3)`? El error de arriba lo dice: `None`, que es el valor que Python devuelve cuando no le dices otra cosa. `None` es un valor de pleno derecho, con su propio tipo, y significa «nada»:

```
def saludar(nombre):  
    print(f"Hola, {nombre}")  
  
resultado = saludar("Ana")  
print(resultado)  
print(type(resultado))
```

```
Hola, Ana  
None  
<class 'NoneType'>
```

Una función sin `return` no está mal escrita. `print` misma es una de ellas. Lo que está mal es esperar un valor de una función que no devuelve ninguno.

`return` termina la función

En cuanto se ejecuta un `return`, la función se acaba y lo que venga después no corre:

```
def signo(x):  
    if x > 0:  
        return "positivo"  
    if x < 0:  
        return "negativo"  
    return "cero"  
  
print(signo(7), signo(-3), signo(0))
```

```
positivo negativo cero
```

Fíjate en que no hacen falta `elif` ni `else`: si la primera condición se cumple, la función ya salió. Este uso de `return` para salir temprano deja el código más plano y más fácil de leer que anidar condicionales.

4. Alcance de las variables

Las variables que nacen dentro de una función viven solo ahí. Cuando la función termina, desaparecen:

```
def calcular():  
    resultado = 42  
  
calcular()  
print(resultado)
```

```
Traceback (most recent call last):
  File "<cell>", line 5, in <module>
    print(resultado)
    ^^^^^^^^^^^
NameError: name 'resultado' is not defined
```

Eso se llama **alcance local**, y no es una limitación sino la razón de ser de las funciones. Puedes escribir una función de cien líneas usando una variable *i* sin preocuparte de si el resto del programa usa otra *i*. Cada función es una caja cerrada.

Al revés sí funciona: desde dentro se puede **leer** una variable de afuera, llamada global.

```
limite = 100

def esta_en_rango(x):
    return 0 < x <= limite

print(esta_en_rango(50))
print(esta_en_rango(200))
```

```
True
False
```

Pero si intentas **asignarle** un valor a esa misma variable global desde dentro, pasa algo que sorprende a todo el mundo la primera vez:

```
contador = 0

def incrementar():
    contador = contador + 1

incrementar()
```

```
Traceback (most recent call last):
  File "<cell>", line 6, in <module>
    incrementar()
    ~~~~~^
  File "<cell>", line 4, in incrementar
    contador = contador + 1
    ^^^^^^^^^
UnboundLocalError: cannot access local variable 'contador' where it is not associated with a value
```

Python decide si una variable es local o global mirando el cuerpo entero de la función antes de ejecutarlo. Como ahí dentro hay una asignación a `contador`, la declara local para toda la función, incluida la línea que intenta leerla. Y esa variable local todavía no tiene valor cuando se la quiere leer.

Existe la palabra `global` para forzar el otro comportamiento. Casi nunca es la solución correcta, y en este curso no la vamos a usar: si una función necesita un valor de afuera, se lo pasas como parámetro, y si produce un valor, lo devuelve con `return`. Ese es el contrato, y respetarlo es lo que hace que una función se pueda entender sin leer el resto del programa.

5. Valores por defecto y argumentos con nombre

Un parámetro puede traer un valor de fábrica, que se usa cuando la llamada no lo menciona:

```
def saludar(nombre, saludo="Hola"):
    print(f"{saludo}, {nombre}")

saludar("Ana")
saludar("Pedro", "Buenas tardes")
```

```
Hola, Ana
Buenas tardes, Pedro
```

Los parámetros con valor por defecto van siempre al final de la lista, porque si no Python no sabría cómo emparejar los argumentos por posición.

Y en la llamada se puede nombrar el parámetro explícitamente, en cuyo caso el orden ya no importa:

```
def rango_valido(x, minimo=0, maximo=100):
    return minimo <= x <= maximo

print(rango_valido(50))
print(rango_valido(50, maximo=10))
print(rango_valido(50, maximo=10, minimo=1))
```

```
True
False
False
```

Esto sirve para dos cosas. Una función con buenos valores por defecto es cómoda en el caso corriente y sigue siendo flexible en el raro. Y una llamada como `rango_valido(50, maximo=10)` se lee sin ir a mirar la definición, cosa que `rango_valido(50, 0, 10)` no.

6. El juego, otra vez

Ahora el proyecto de la clase. Vamos a tomar el juego de adivinar tal como quedó la semana pasada y reescribirlo en funciones. La pregunta que guía el trabajo es siempre la misma: **¿puedo explicar lo que hace este pedazo en una sola frase?** Si la respuesta es sí, ese pedazo es una función y esa frase es su nombre.

En el programa de la clase pasada hay tres frases así. Pedirle un número al jugador. Decirle si se pasó o se quedó corto. Jugar una partida completa.

La primera pide un número y se asegura de que esté en el rango:

```
def pedir_intento(minimo, maximo):
    """Pide un número al usuario hasta que escriba uno dentro del rango."""
    while True:
        intento = int(input(f"Adivina mi número ({minimo}-{maximo}): "))

        if minimo <= intento <= maximo:
            return intento

    print(f"Debe estar entre {minimo} y {maximo}")
```

No se limita a leer: repite hasta que el número sirva. Es el `while True` con `break` de la clase pasada, y aquí queda más limpio porque `return` sale del ciclo y de la función a la vez.

La segunda es la más corta, y es la única que no devuelve nada, porque su trabajo entero es escribir en la pantalla:

```
def dar_pista(intento, secreto):
    """Le dice al jugador si el número secreto es mayor o menor que su intento."""
    if intento < secreto:
        print("Mi número es mayor")
    else:
        print("Mi número es menor")
```

Y la tercera usa a las otras dos:

```
import random

def jugar_partida(minimo=1, maximo=100, max_intentos=7):
    """Juega una partida completa. Devuelve True si el jugador adivinó."""
    secreto = random.randint(minimo, maximo)

    for numero_de_intento in range(1, max_intentos + 1):
        intento = pedir_intento(minimo, maximo)

        if intento == secreto:
            print(f"¡Acertaste en {numero_de_intento} intentos!")
            return True

        dar_pista(intento, secreto)

    print(f"Se acabaron los intentos. Era {secreto}")
    return False
```

Dos detalles. Cuenta los intentos con un `for` sobre `range` en lugar de la variable `intentos` que llevábamos a mano, porque el número de vueltas se conoce de antemano y esa es la regla de la clase pasada. Y **devuelve** si el jugador ganó en vez de imprimirlo, que es lo que permite escribir el programa principal así:

```
if jugar_partida():
    print("Ganaste")
else:
    print("Perdiste")
```

Cuatro líneas. Si `jugar_partida` imprimiera el resultado en lugar de devolverlo, ni este `if` ni el torneo de la próxima sección serían posibles.

Los docstrings

Ese texto entre comillas triples en la primera línea del cuerpo se llama **docstring**, y es la forma normal de decir qué hace una función. No es un comentario: Python lo guarda y lo puede mostrar.

```
def area_circulo(radio):
    """Devuelve el área de un círculo del radio dado."""
    return 3.1416 * radio ** 2

print(area_circulo.__doc__)
```

```
Devuelve el área de un círculo del radio dado.
```

La misma función `help` que puedes usar en la consola con `help(print)` lee esos docstrings. Escríbelos en una línea y en presente: «devuelve», «calcula», «pide».

7. El torneo

Ahora sí, lo que la clase pasada era imposible. Tres partidas y un marcador:

```
victorias = 0

for numero_de_partida in range(1, 4):
    print(f"--- Partida {numero_de_partida} de 3 ---")

    if jugar_partida():
        victorias += 1

print(f"Ganaste {victorias} de 3 partidas")
```

Seis líneas, y ninguna de ellas sabe nada de números al azar ni de pistas. Eso es lo que compran las funciones: el torneo se escribe pensando en partidas, no en intentos.

Y como `jugar_partida` tiene valores por defecto, subir la dificultad en cada ronda no cuesta nada:

```
victorias = 0

for numero_de_partida in range(1, 4):
    tope = 10 ** numero_de_partida
    print(f"--- Partida {numero_de_partida}: del 1 al {tope} ---")

    if jugar_partida(maximo=tope):
        victorias += 1

print(f"Ganaste {victorias} de 3 partidas")
```

```
$ python torneo.py
--- Partida 1: del 1 al 10 ---
Adivina mi número (1-10): 5
Mi número es mayor
Adivina mi número (1-10): 8
¡Acertaste en 2 intentos!
--- Partida 2: del 1 al 100 ---
Adivina mi número (1-100): 50
...
```

La primera ronda va del 1 al 10, la segunda al 100 y la tercera al 1000, y el 7 de `max_intentos` se quedó donde estaba. Esa llamada, `jugar_partida(maximo=tope)`, se entiende sola.

Queda un detalle honesto: siete intentos alcanzan de sobra para cien números, pero para mil hacen falta diez. Arréglalo, y fíjate en que el arreglo es **una sola línea en una sola función**. Ese es el argumento entero de la clase de hoy.

8. Resumen

- `def` define una función; los paréntesis la llaman. Definir no ejecuta.
- Los **parámetros** están en la definición, los **argumentos** en la llamada.
- `return` devuelve un valor y termina la función. Una función con `return` es una expresión y se puede usar dentro de otra cuenta.
- Una función que calcula devuelve; imprimir es trabajo del programa principal.
- Una función sin `return` devuelve `None`.

- Las variables de dentro no se ven desde fuera. Desde dentro se puede leer una global, pero asignarle crea una local y da error.
- Los parámetros pueden traer valor por defecto, y en la llamada se pueden nombrar.
- Descomponer es buscar los pedazos que se explican en una frase. Esa frase es el nombre de la función.

Ejercicios

1. Escribe `es_primo(n)` que devuelva `True` o `False`. Después úsala para imprimir todos los primos menores que 100. Cuida los casos $n = 0$, $n = 1$ y los negativos.
2. Escribe `mcd(a, b)`, el máximo común divisor, con el algoritmo de Euclides: el mcd de a y b es el mcd de b y el resto de dividir a entre b , hasta que el resto sea cero. Con ella escribe `mcm(a, b)` en una sola línea.
3. Escribe `factorial(n)` y úsala para escribir `combinaciones(n, k)`, que calcula de cuántas maneras se pueden escoger k elementos de un conjunto de n . La fórmula es $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Comprueba que `combinaciones(5, 2)` da 10.
4. Escribe `longitud_collatz(n)`, que devuelve cuántos pasos tarda la secuencia de Collatz de la clase pasada en llegar a 1. Úsala para encontrar el número menor que 1000 con la secuencia más larga.
5. Escribe `dibujar_triangulo(altura, caracter="*")` que imprima un triángulo. Con `dibujar_triangulo(4)` debe salir:

```
*
**
***
****
```

Añádele un parámetro `invertido=False` que lo dibuje al revés cuando valga `True`.

6. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

(a)

```
def f(x):
    x = x + 1
    return x

y = 5
f(y)
print(y)
```

(b)

```
def g(n):
    if n > 0:
        return "positivo"
    print("no era positivo")

print(g(5))
print(g(-5))
```

(c)

```
def h(a, b=2, c=3):
    return a * 100 + b * 10 + c
```

```
print(h(1))  
print(h(1, 5))  
print(h(1, c=5))
```